

Altair

Python Visualization Library

Mark & Channel

→ Points



→ Lines



→ Areas



→ Position

→ Horizontal



→ Vertical



→ Both



→ Color



→ Shape



→ Tilt



→ Size

→ Length



→ Area



→ Volume



Data Visualization

Data, Mark, Encoding

- **Pandas DataFrame**을 가장 많이 사용합니다.
 - Names data fields (column)으로 구성
- 필요하다면 url로부터 data를 로드하는 것도 가능합니다.
 - json, csv 형식의 파일 url

- 실습에서는 **vega-datasets** repo의 dataset을 사용할 예정

```
from vega_datasets import data  
data.list_datasets()
```

- 자동차, 인구, 국가별 경제/의료 수준 데이터를 불러옵니다.

```
cars = data.cars()  
pop = data.population()  
gapminder = data.gapminder()
```

```
data2000 = gapminder.loc[gapminder['year']==2000]  
dataUS = gapminder.loc[gapminder["country"] == "United States"]
```

- 아래와 같이 Data가 정상적으로 출력되셨나요?

```
data2000.head(5)
```

	year	country	cluster	pop	life_expect	fertility
9	2000	Afghanistan	0	23898198	42.129	7.4792
20	2000	Argentina	3	37497728	74.340	2.3500
31	2000	Aruba	3	69539	73.451	2.1240
42	2000	Australia	4	19164620	80.370	1.7560
53	2000	Austria	1	8113413	78.980	1.3820

- Chart object는 Altair에서 가장 기본이 되는 object입니다.
- Data frame을 single argument로 받을 수 있게 해줍니다.

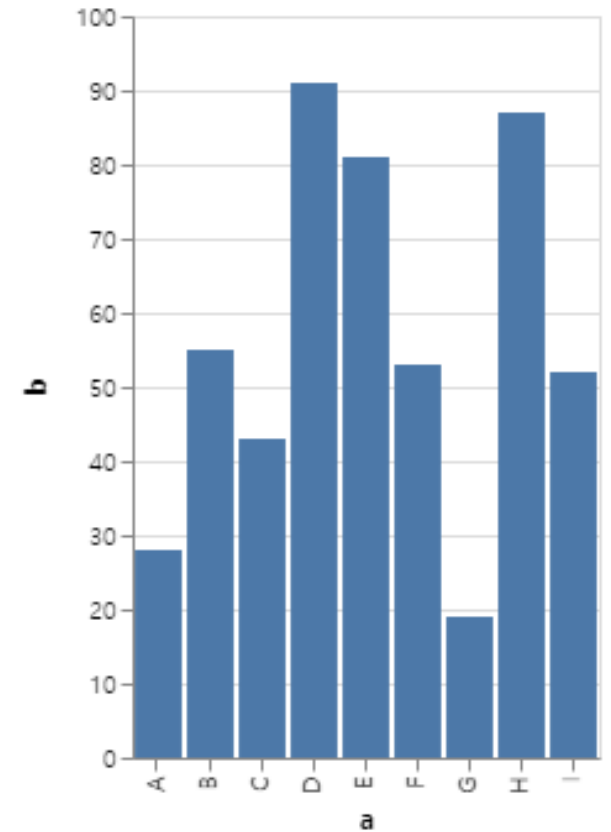
```
source = pd.DataFrame({  
    'a': ['A', 'B', 'C', 'D', 'E', 'G', 'H', 'I'],  
    'b': [28, 55, 43, 91, 81, 53, 19, 87, 52]  
})
```

```
alt.Chart(source).mark_bar().encode(  
    x='a:N',  
    y='b'  
)
```

- Chart object는 Altair에서 가장 기본이 되는 object입니다.
- Data frame을 single argument로 받을 수 있게 해줍니다.

```
source = pd.DataFrame({  
    'a': ['A', 'B', 'C', 'D', 'E', 'G', 'H', 'I'],  
    'b': [28, 55, 43, 91, 81, 53, 19, 87, 52]  
})
```

```
alt.Chart(source).mark_bar().encode(  
    x='a:N',  
    y='b'  
)
```



- Channel : Mark의 모양을 결정짓는 요소 (position, shape, color..)



1:
vertical position

mark: line



2:
vertical position
horizontal position

mark: point



3:
vertical position
horizontal position
color hue

mark: point



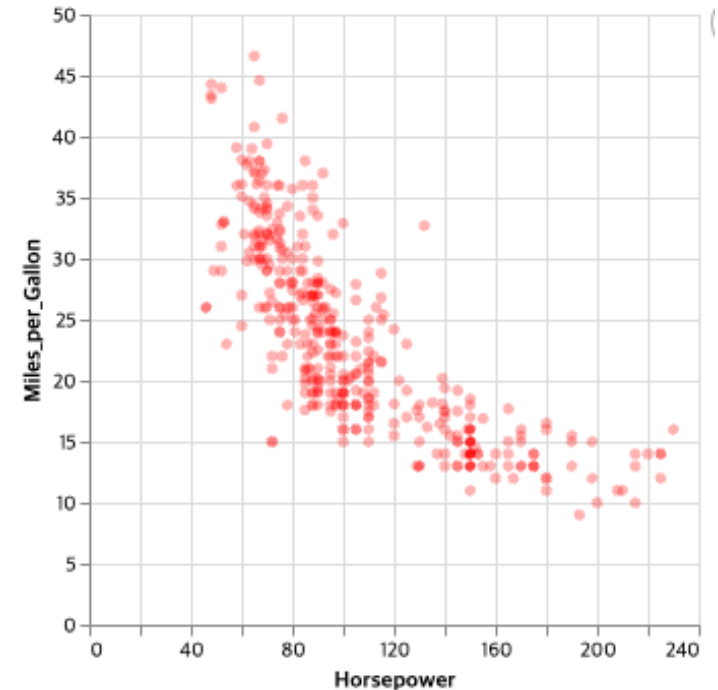
4:
vertical position
horizontal position
color hue
size (area)

mark: point

- Altair에서는 encode() method를 통해 사용할 수 있습니다.

```
cars = data.cars()

alt.Chart(cars).mark_circle(
    color='red',
    opacity= 3
).encode(
    x='Horsepower:Q',
    y='Miles_per_Gallon:Q'
)
```



- 각 **channel (encoding)**에 대응되는 데이터 column이 어떤 종류인지 명시해줘야합니다.
 - 연속/이산변수, 숫자형/범주형으로 나눌 수 있습니다.

```
cars = data.cars()

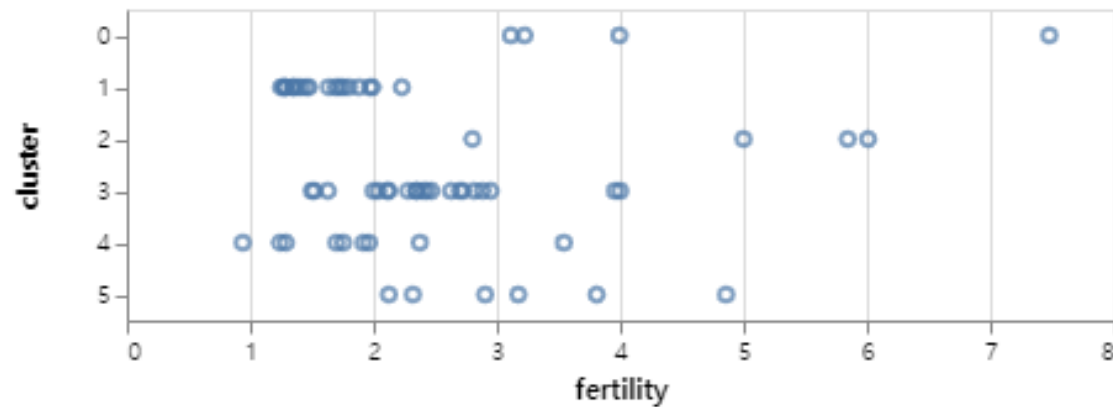
alt.Chart(cars).mark_circle(
    color='red',
    opacity=.3
).encode(
    x='Horsepower:Q',
    y='Miles_per_Gallon:Q'
)
```

Data Type	Shorthand Code	Description
quantitative	Q	a continuous real-valued quantity
ordinal	O	a discrete ordered quantity
nominal	N	a discrete unordered category
temporal	T	a time or date value
geojson	G	a geographic shape

Position (X, Y)

- 기본적으로 축을 나누는 것과 title은 자동으로 결정됩니다.
 - X축은 Quantitative data (fertility)에
 - Y축에는 Ordinal data (cluster)를 나타냅니다.

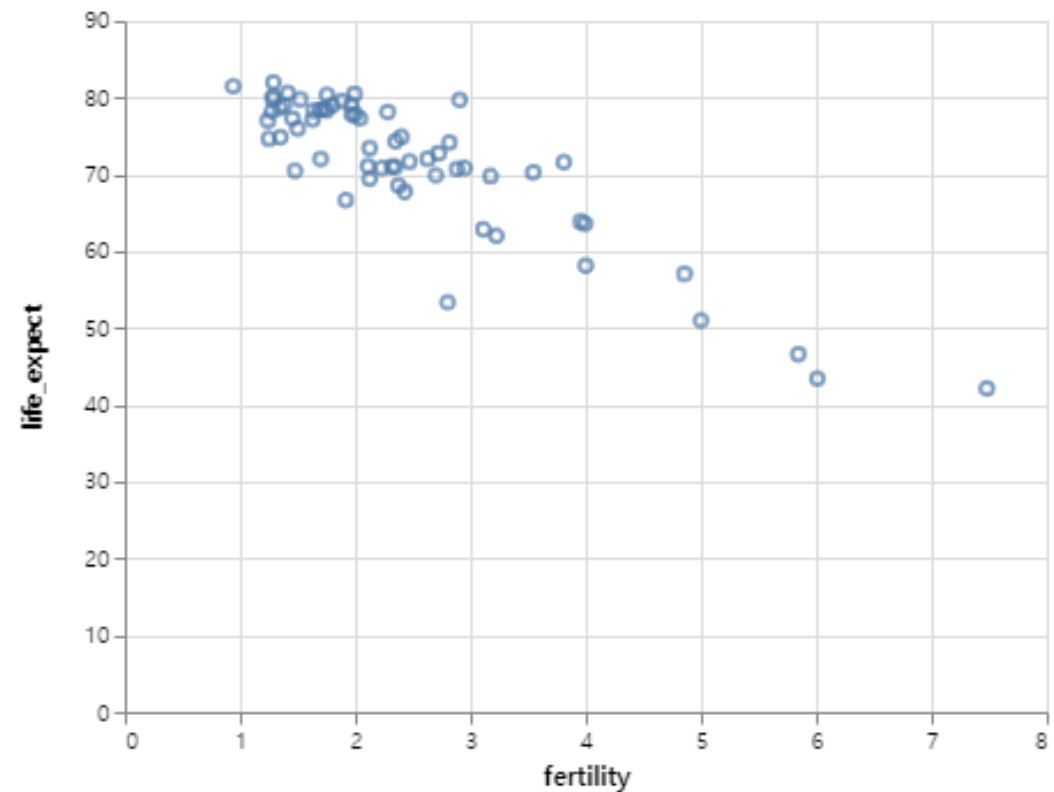
```
alt.Chart(data2000).mark_point().encode(  
  alt.X('fertility:Q'),  
  alt.Y('cluster:O')  
)
```



Position (X, Y)

- Y축의 값을 Quantitative data (life_expect)로 바꿔봅시다.
 - 출산율 / 기대수명간의 관계를 알 수 있습니다.

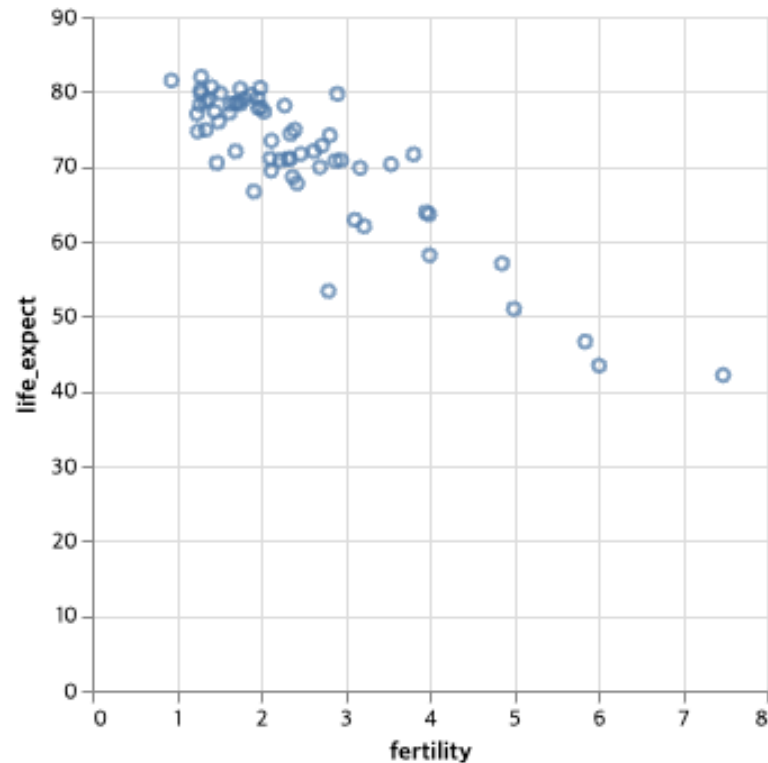
```
alt.Chart(data2000).mark_point().encode(  
  alt.X('fertility:Q'),  
  alt.Y('life_expect:Q')  
)
```



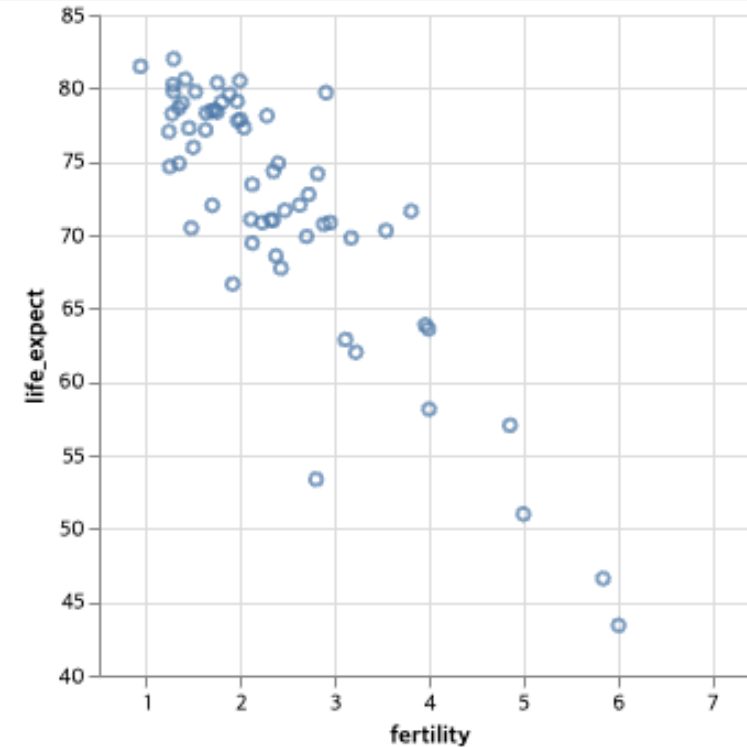
Position (X, Y)

- X축, y축의 범위를 조절해서 분포를 자세히 관찰할 수 있습니다.

```
alt.Chart(data2000).mark_point().encode(  
  alt.X('fertility:Q'),  
  alt.Y('life_expect:Q')  
)
```



```
alt.Chart(data2000).mark_point().encode(  
  alt.X('fertility:Q', scale=alt.Scale(zero=False)),  
  alt.Y('life_expect:Q', scale=alt.Scale(zero=False))  
)
```



Position (X, Y)

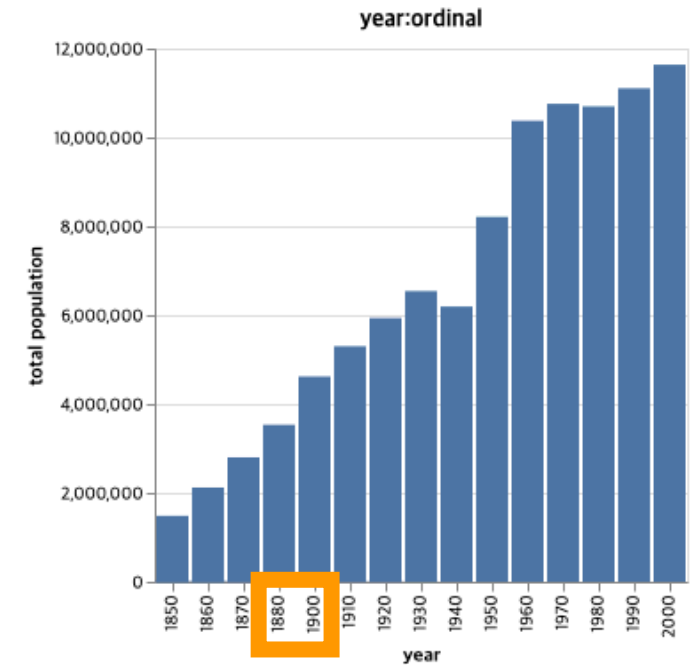
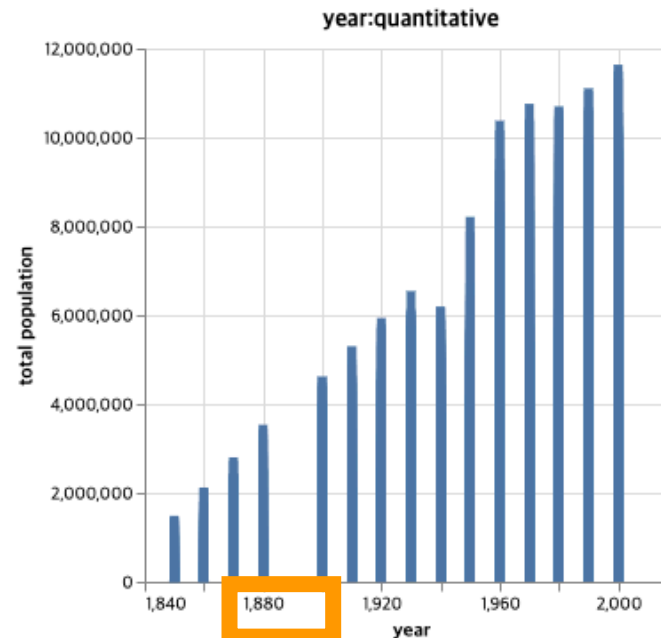
- 연도는 숫자이므로 Quantitative type이 되어야 합니다.
 - Type을 잘못 명시하면 잘못된 정보를 전달합니다

```
pop = data.population()

chart_quantitative = alt.Chart(pop).mark_bar().encode(
    alt.('year:Q'),
    alt.('mean(people):Q', title="total population"),
    tooltip=['year', 'people']
).properties(
    title='year:quantitative'
)

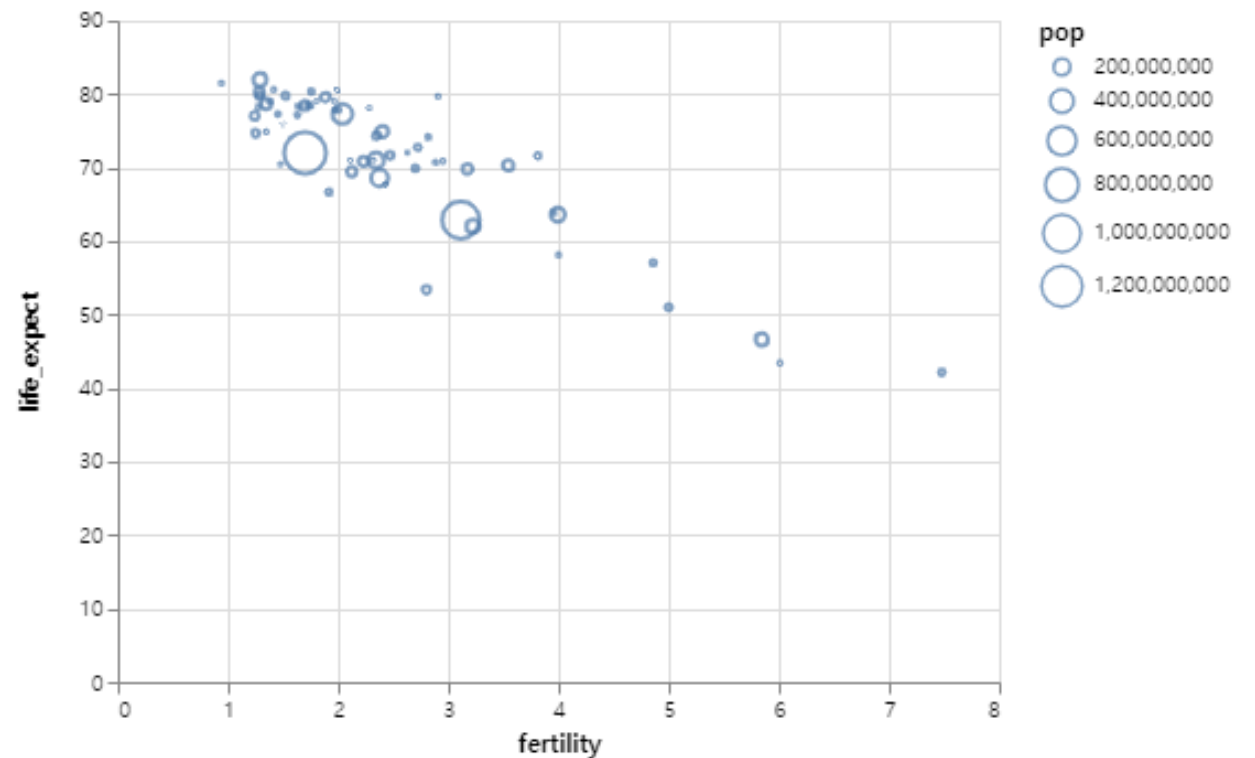
chart_ordinal = alt.Chart(pop).mark_bar().encode(
    alt.('year:O'),
    alt.('mean(people):Q', title="total population"),
    tooltip=['year', 'people']
).properties(
    title='year:ordinal'
)

chart_quantitative | chart_ordinal
```



- Mark의 종류에 따라 size의 정의가 다를 수 있습니다.
 - 앞에서 그린 scatterplot에 size encoding을 추가해봅시다.

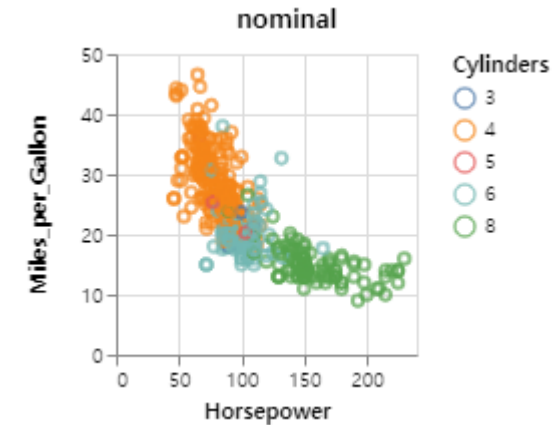
```
alt.Chart(data2000).mark_point().encode(  
  alt.X('fertility:Q'),  
  alt.Y('life_expect:Q'),  
  alt.Size('pop:Q')  
)
```



- Color를 설정하는 방식은 data type에 따라 달라집니다.

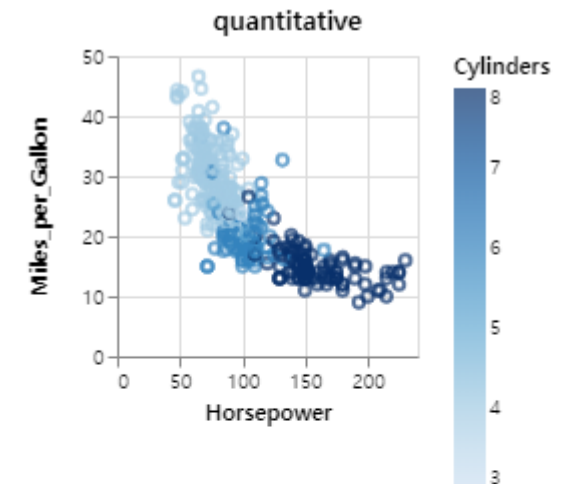
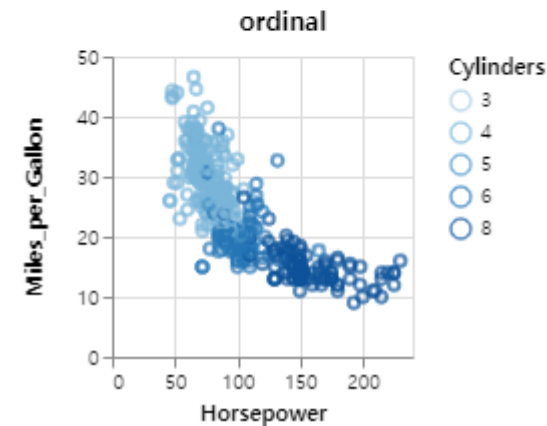
- **Nominal** 의 경우

- 시각적으로 잘 구분되도록

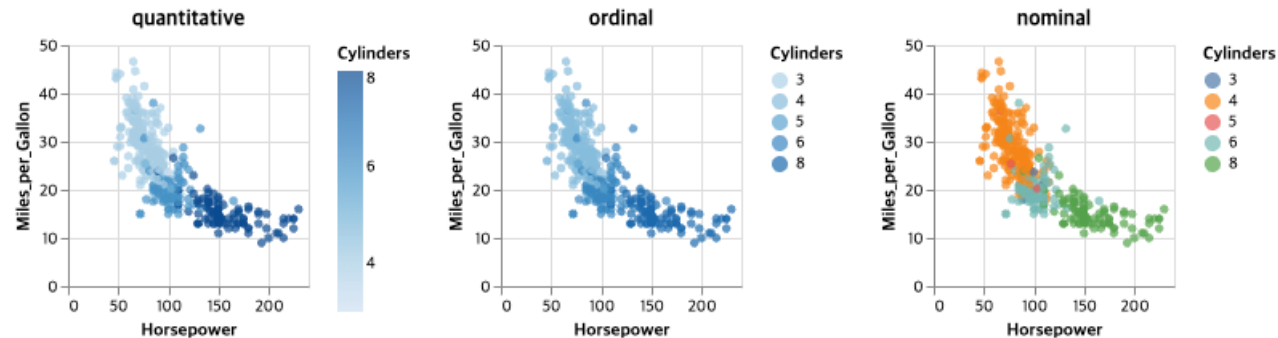


- **Ordinal** and **quantitative** 의 경우

- 순서가 드러나도록
 - Ordered color gradients를 사용

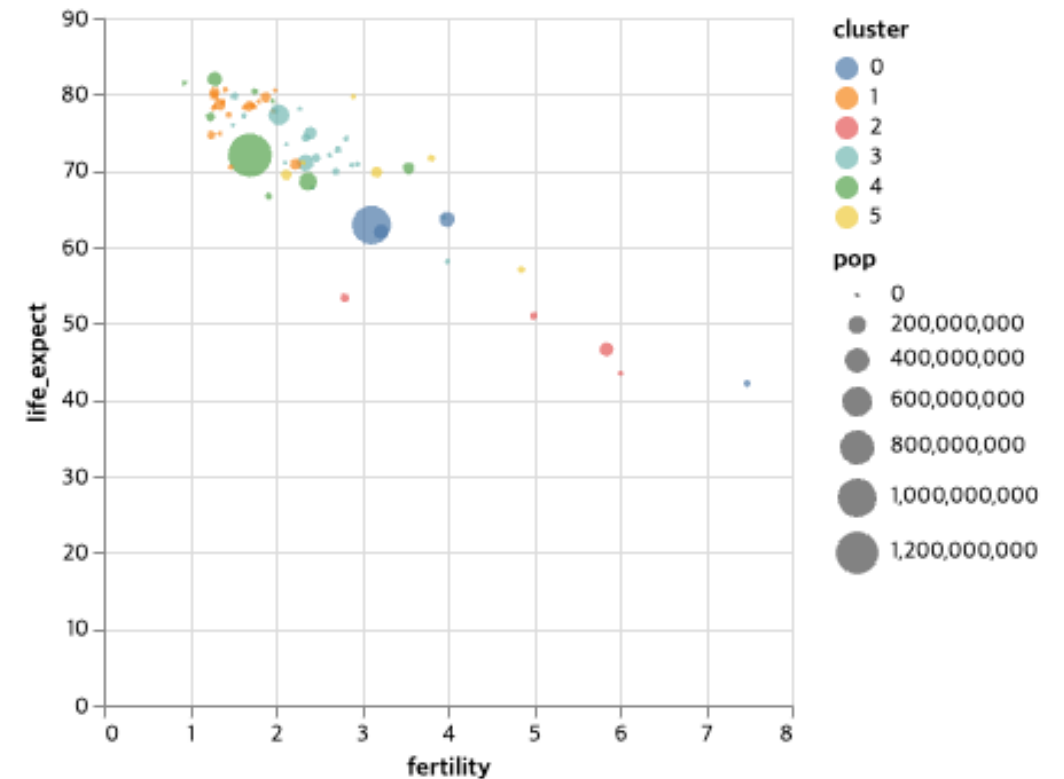


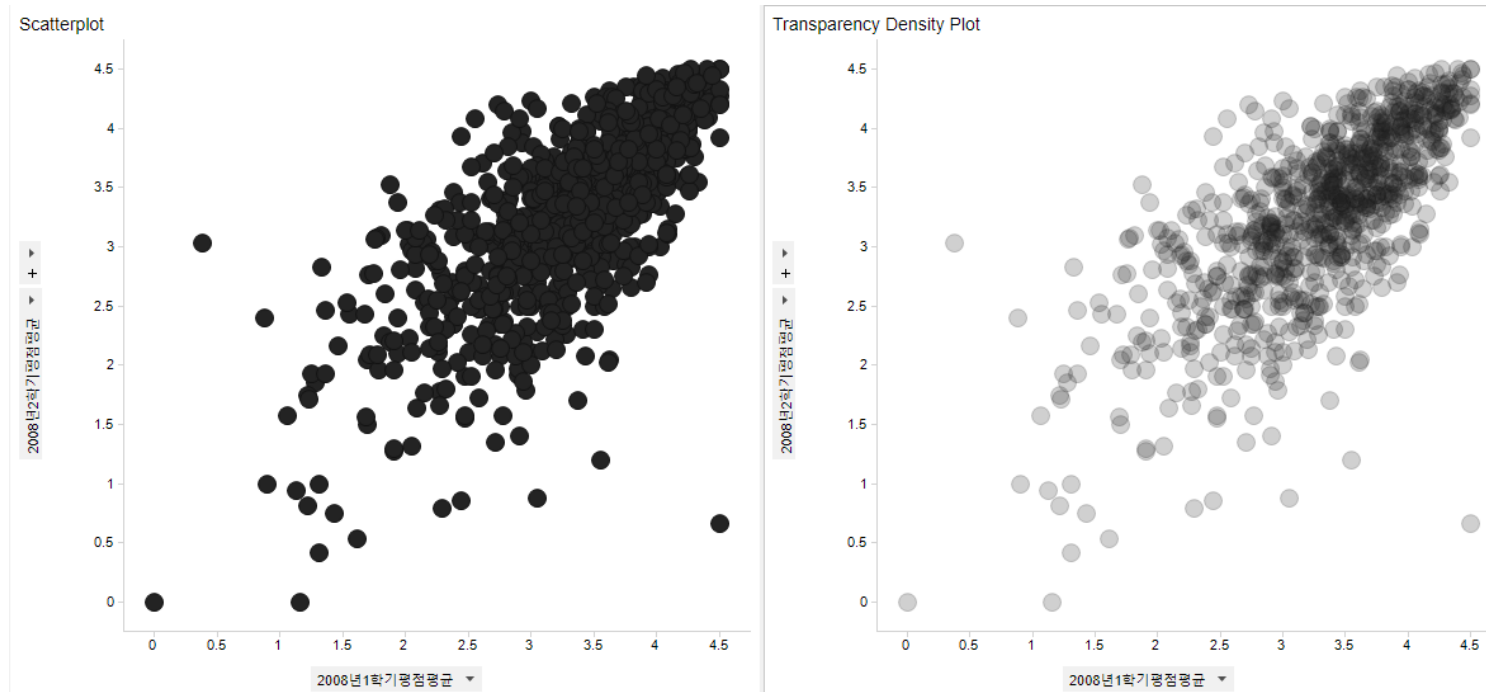
```
base = alt.Chart(cars).mark_circle().encode(  
    alt.X('Horsepower:Q'),  
    alt.Y('Miles_per_Gallon:Q'),  
  
).properties(  
    width=150,  
    height=150  
)  
  
alt.hconcat(  
    base.encode(color='Cylinders:Q').properties(title='quantitative'),  
    base.encode(color='Cylinders:O').properties(title='ordinal'),  
    base.encode(color='Cylinders:N').properties(title='nominal')  
)
```



- X, Y, Size 인코딩을 사용한 scatterplot에 Color를 추가해봅시다.

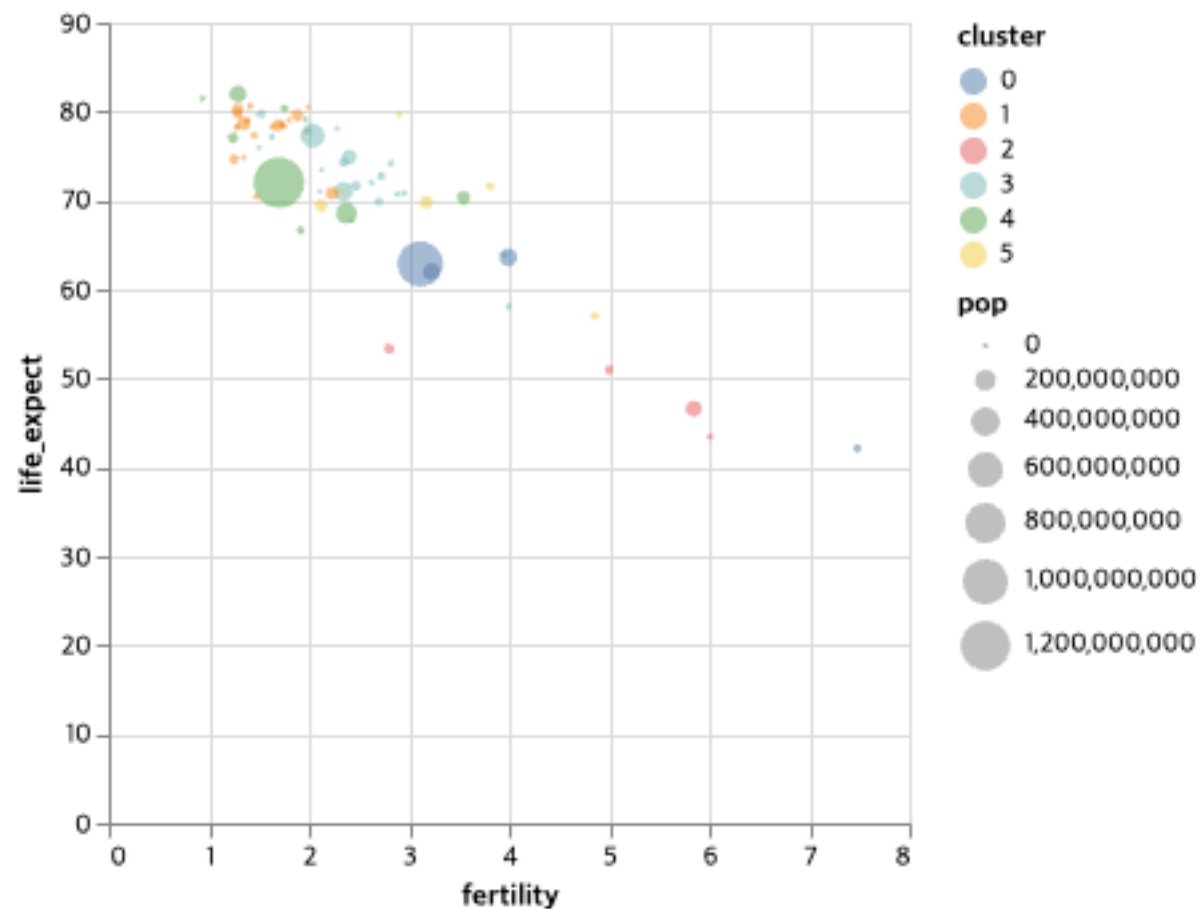
```
alt.Chart(data2000).mark_point(filled=True).encode(
    alt.X('fertility:Q'),
    alt.Y('life_expect:Q'),
    alt.Size('pop:Q'),
    alt.Color('cluster:N')
)
```





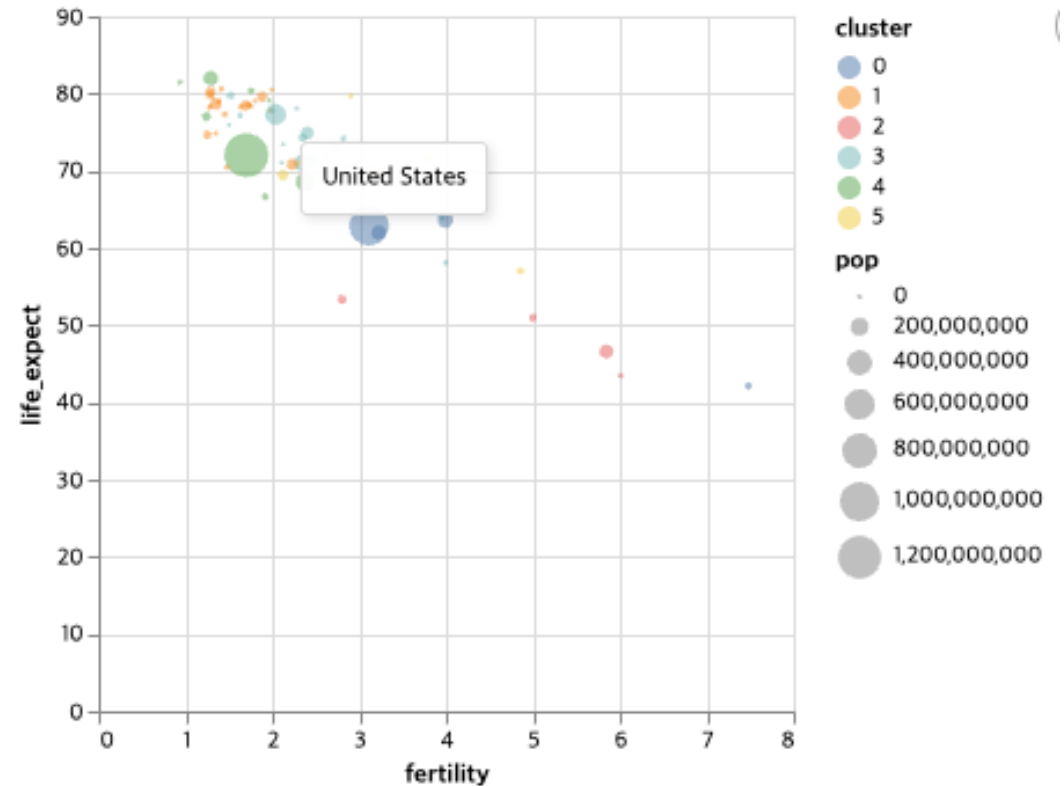
- Over-plotting 문제를 해결하기 위해 투명도 값을 조절합니다

```
alt.Chart(data2000).mark_point(filled=True).encode(  
  alt.X('fertility:Q'),  
  alt.Y('life_expect:Q'),  
  alt.Size('pop:Q'),  
  alt.Color('cluster:N'),  
  alt.OpacityValue(0.5)  
)
```



- 많은 Encoding들을 시각화에 적용했지만 아직 각각의 포인트들이 어느 나라에 대응하는지 알 수가 없습니다
- Tooltip 인코딩을 통해 각 점에 대한 설명을 차트에 추가합니다.

```
alt.Chart(data2000).mark_point(filled=True).encode(  
  alt.X('fertility:Q'),  
  alt.Y('life_expect:Q'),  
  alt.Size('pop:Q'),  
  alt.Color('cluster:N'),  
  alt.OpacityValue(0.5),  
  alt.Tooltip('country:N')  
)
```



- 이번에는 Altair의 마크를 다루어 봅시다.

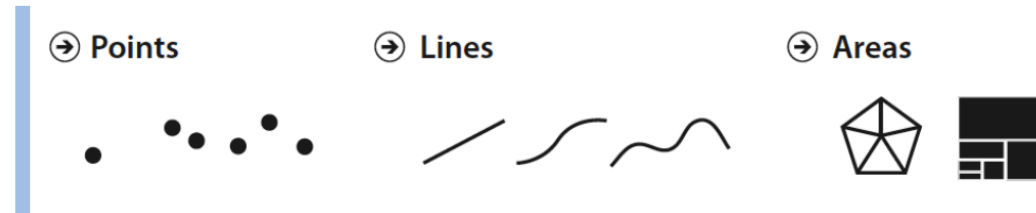
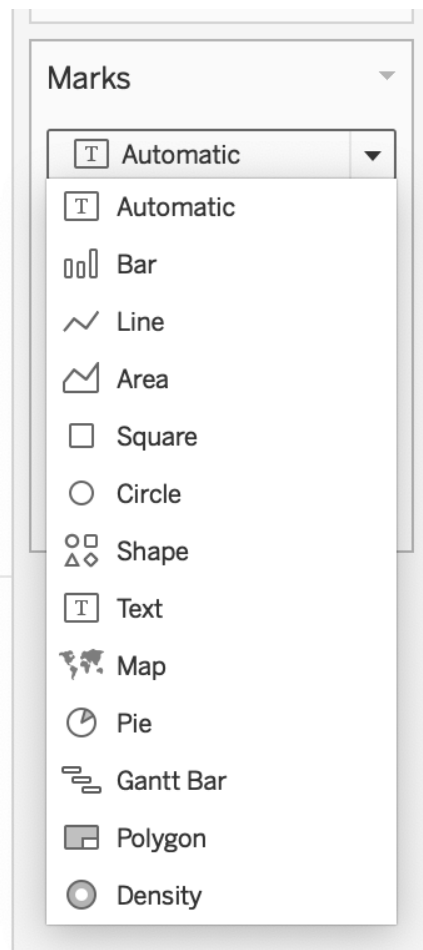
```
import altair as alt
from vega_datasets import data

url = data.cars.url

alt.Chart(url).mark_circle(
    color='red',
    opacity=0.3
).encode(
    x='Horsepower:Q',
    y='Miles_per_Gallon:Q'
)
```

• Geometric Primitives

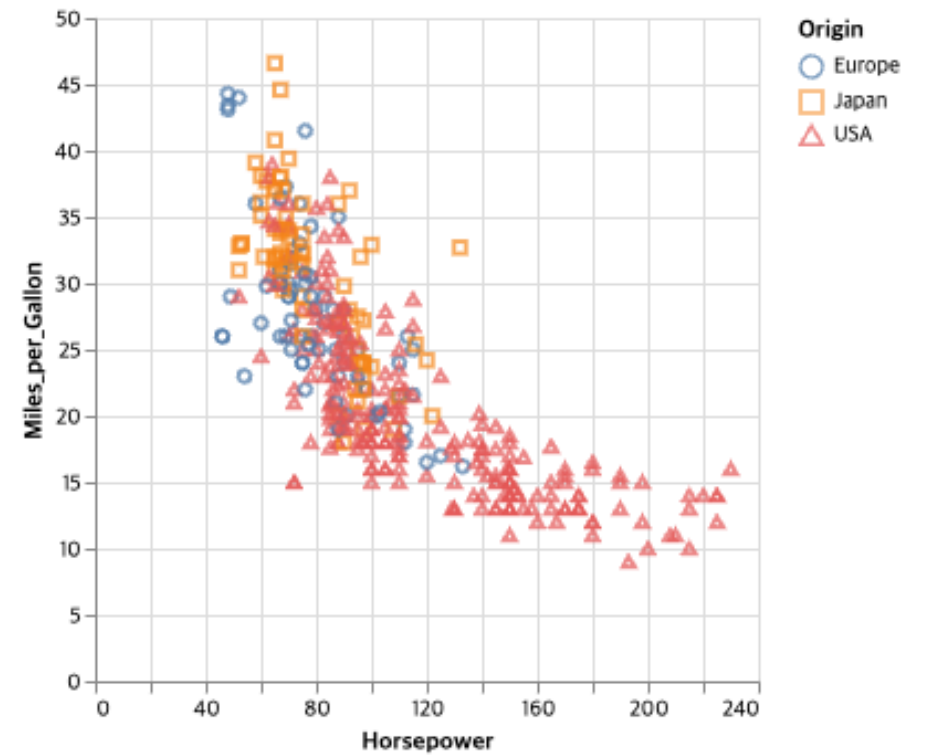
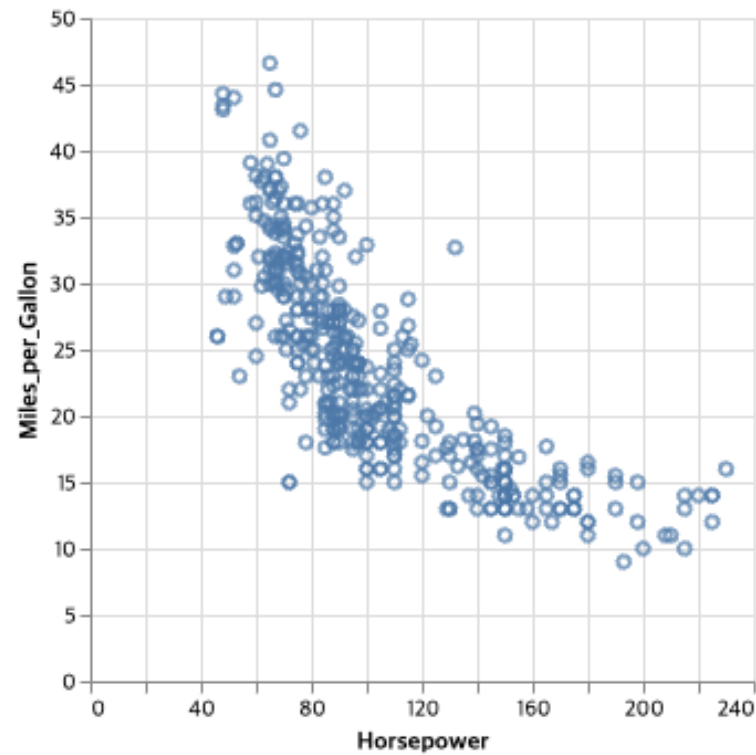
- area
- bar
- circle
- geoshape
- line
- point
- rect
- rule
- square
- text
- tick



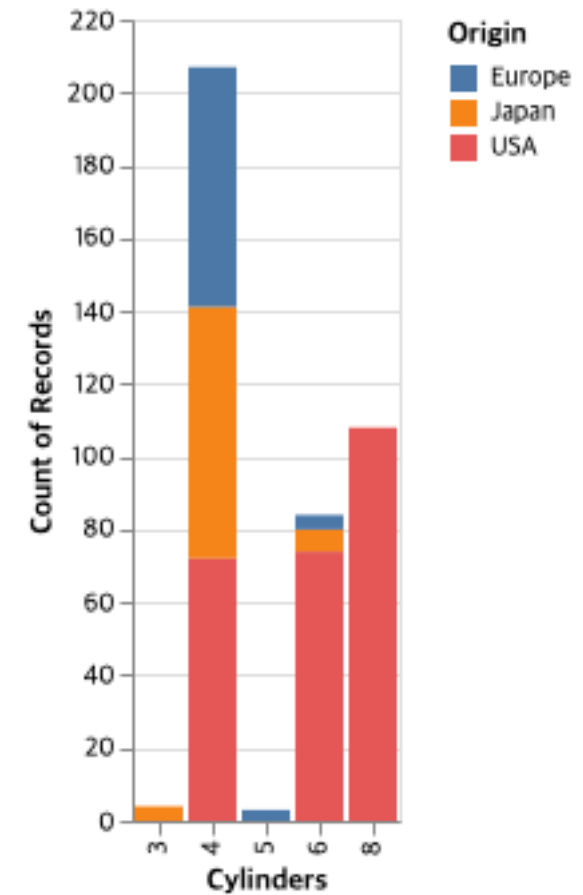
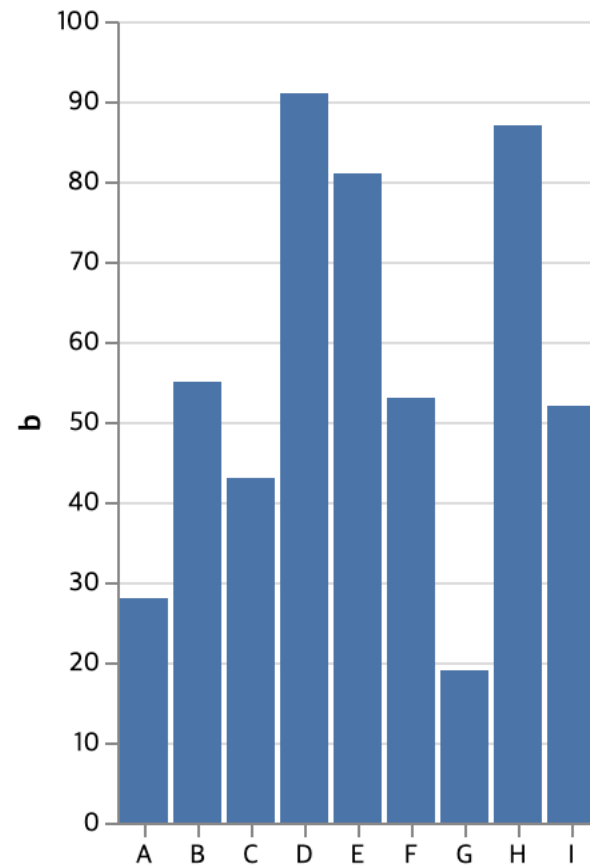
- 점 하나로 데이터를 나타냅니다.

- 사용 가능한 채널

- x
- y
- color
- size
- shape

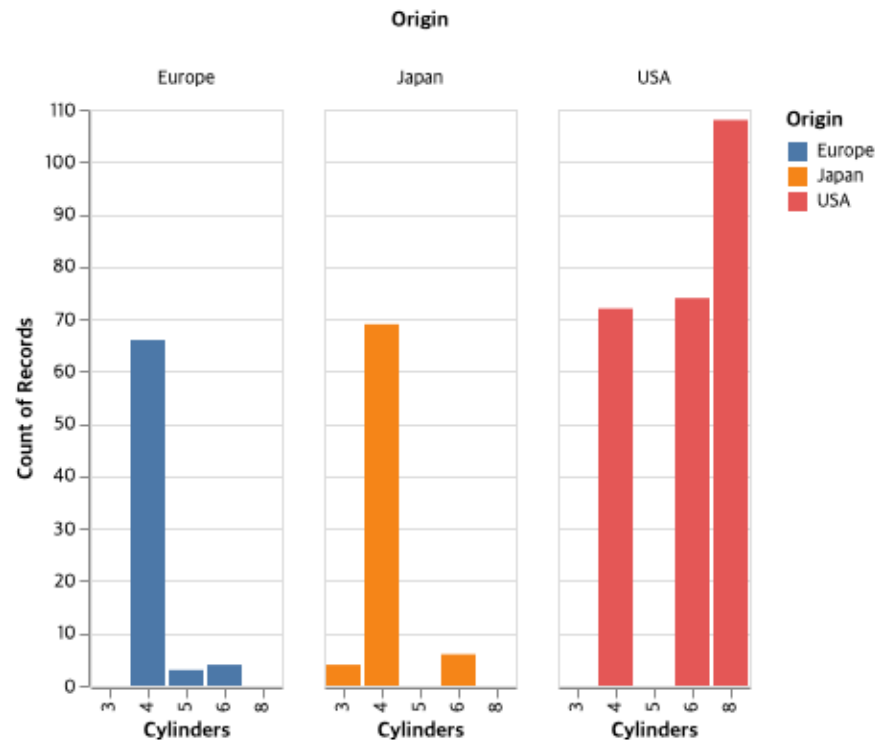
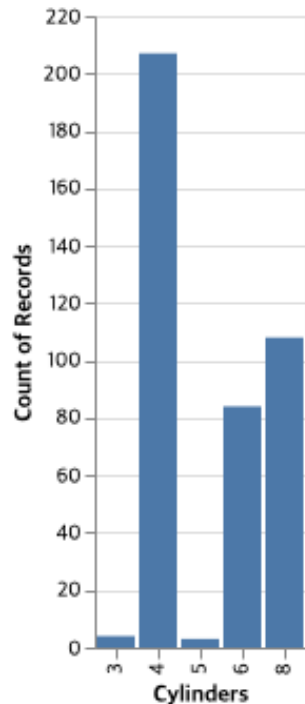


- 데이터를 막대로 표현합니다.
- 사용 가능한 채널
 - x
 - y
 - color

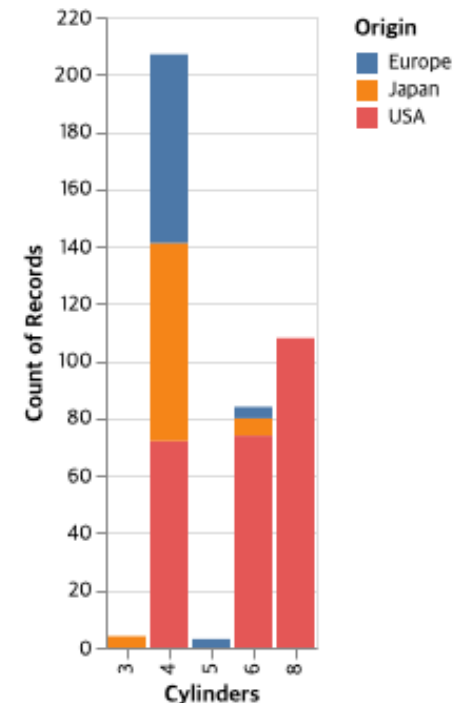


- Color인코딩만 추가해서 누적 막대 그래프를 그릴 수 있습니다.

```
alt.Chart(cars).mark_bar().encode(
  x='Cylinders:Q',
  y='count():Q',
)
```

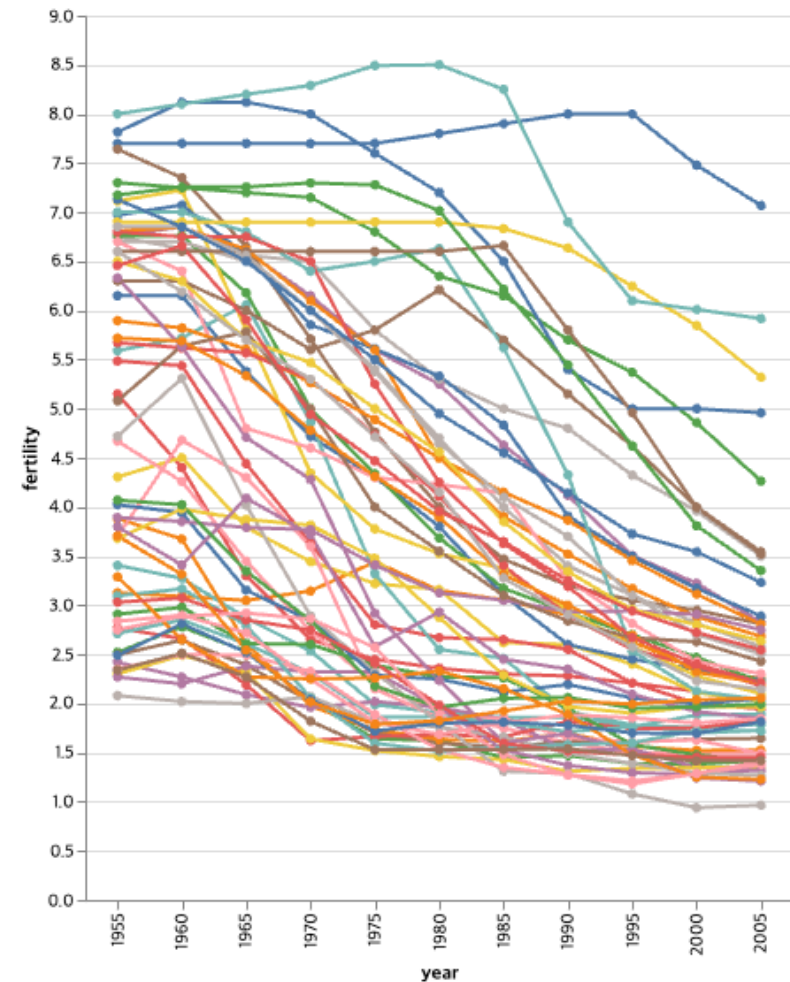


```
alt.Chart(cars).mark_bar().encode(
  x='Cylinders:Q',
  y='count():Q',
  color='Origin:N'
)
```



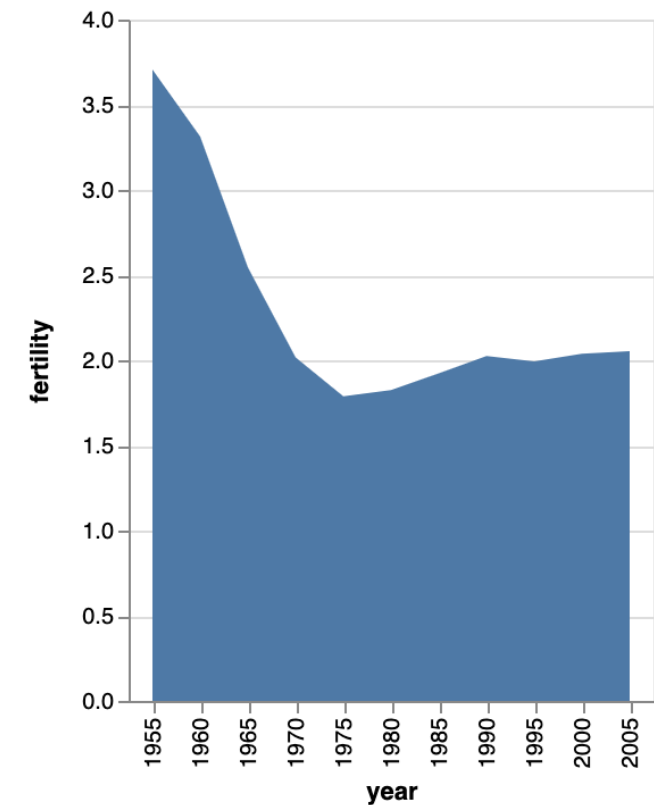
- 선 마크는 포인트 마크를 사이를 선으로 잇습니다.

```
alt.Chart(gapminder).mark_line(point=True).encode(  
    alt.X('year:Q'),  
    alt.Y('fertility:Q'),  
    alt.Color('country:N', legend=None),  
    alt.Tooltip('country:N')  
) .properties(width=400, height=500)
```

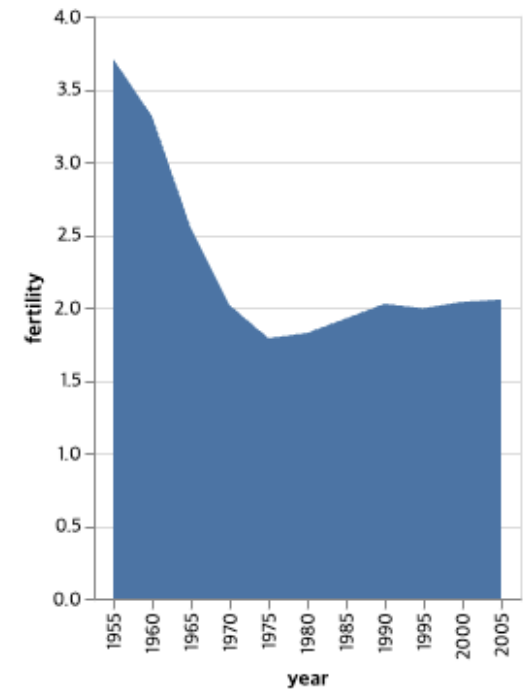
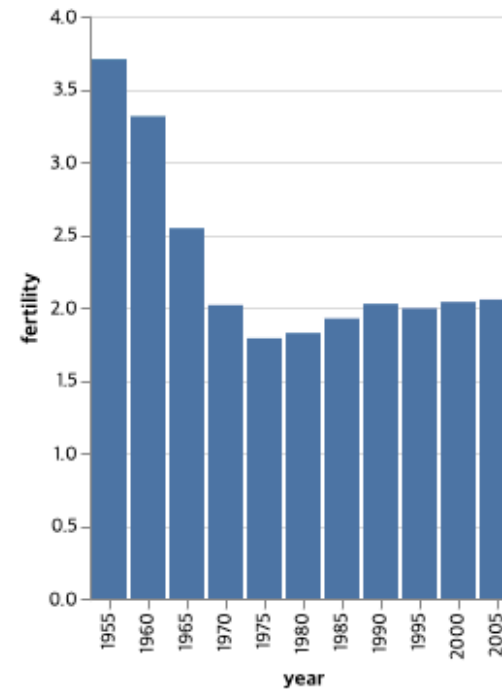
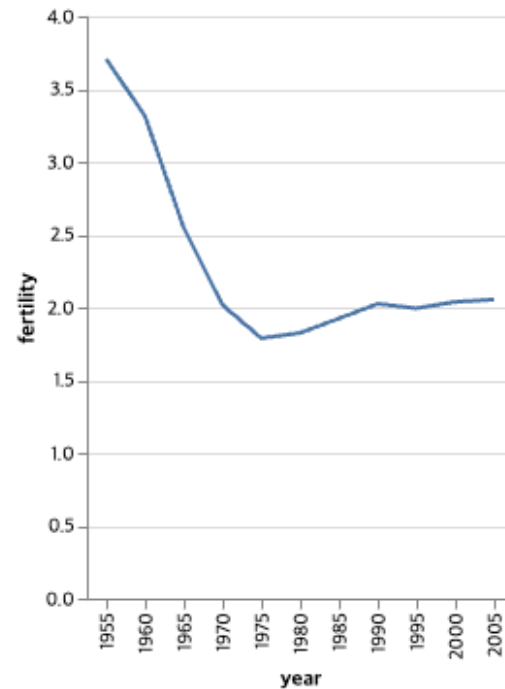
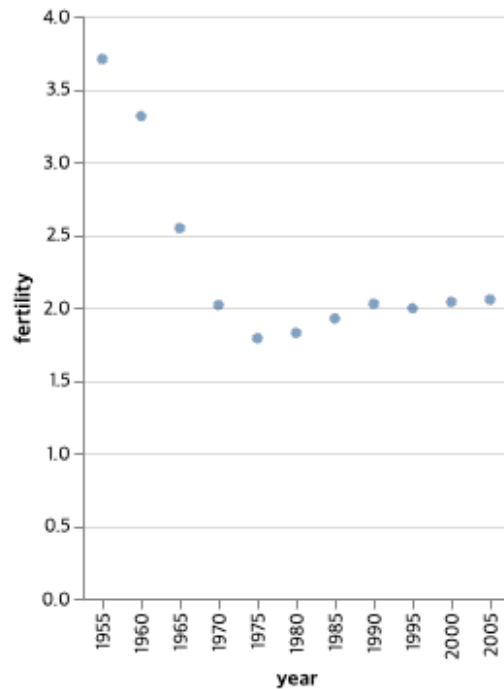


- 막대 마크와 선 마크를 합친 것입니다.
- 막대 마크처럼 아래 영역에 색을 칠하고, 쌓을 수 있습니다.

```
dataUS = data.loc[data['country'] == 'United States']  
  
alt.Chart(dataUS).mark_area().encode(  
    alt.X('year:O'),  
    alt.Y('fertility:Q')  
)
```



- 동일한 인코딩, 데이터로 Point, Line, Bar, Area 마크를 사용하여 차트를 그릴 수 있습니다.
- Area 마크도 Line 마크처럼 기울기를 이용하여 데이터의 변화를 나타냅니다.



Data Visualization

Composition

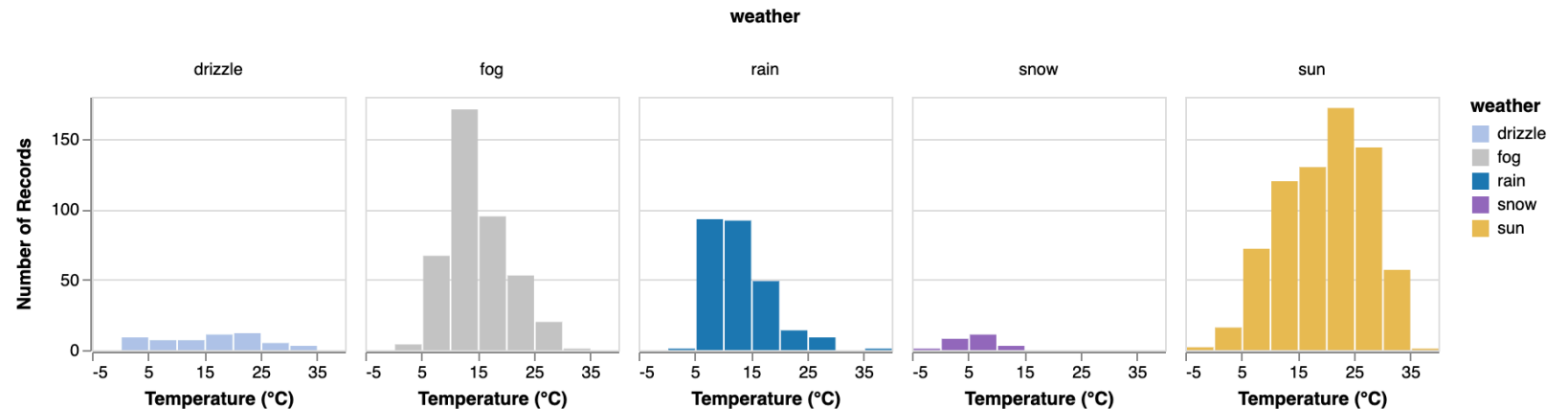
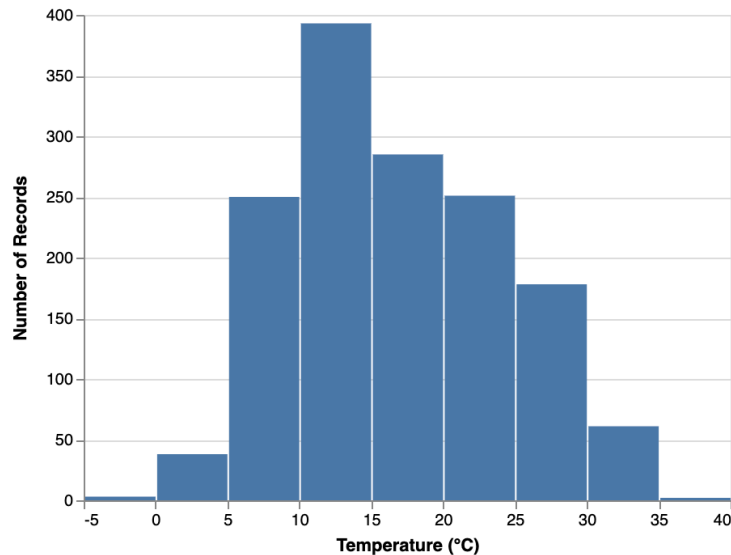
- Weather Dataset을 탐색해봅시다.

- 위치 (뉴욕 또는 시애틀)
- 날짜, 강수량
- 최대온도
- 최소온도
- 바람세기
- 날씨 (이슬비, 안개, 맑음, 눈 등)

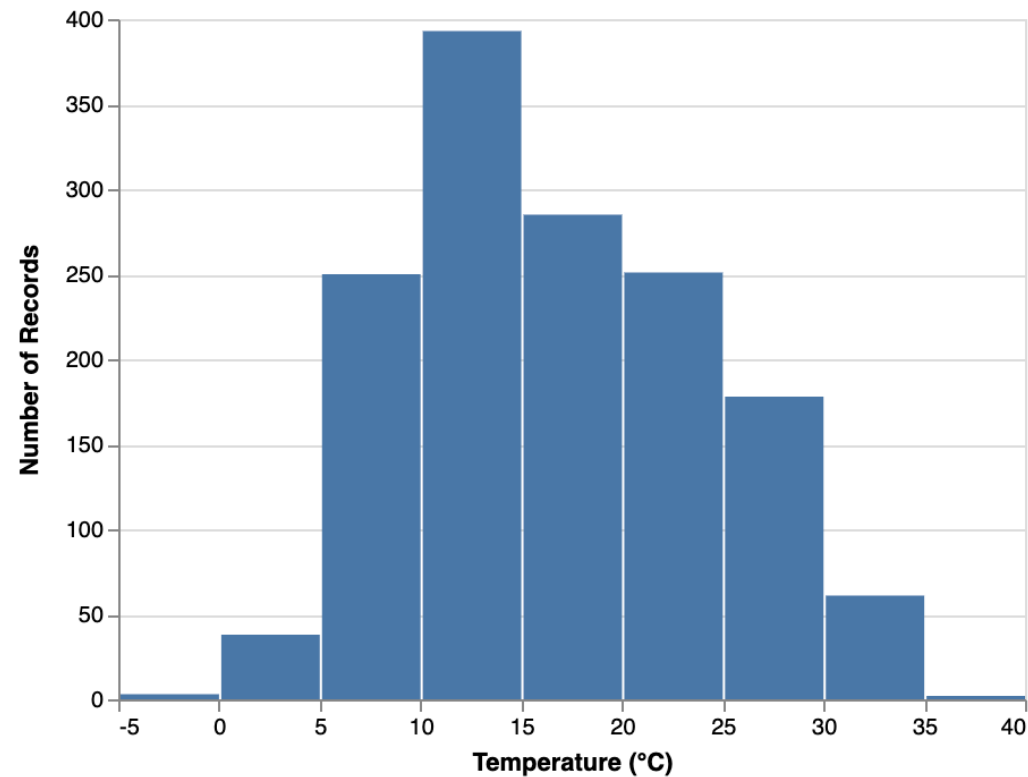
```
weather_url = 'https://vega.github.io/vega-datasets/data/weather.csv'  
  
weather = pd.read_csv(weather_url)  
weather.head()
```

	location	date	precipitation	temp_max	temp_min	wind	weather
0	Seattle	2012-01-01	0.0	12.8	5.0	4.7	drizzle
1	Seattle	2012-01-02	10.9	10.6	2.8	4.5	rain
2	Seattle	2012-01-03	0.8	11.7	7.2	2.3	rain
3	Seattle	2012-01-04	20.3	12.2	5.6	4.7	rain
4	Seattle	2012-01-05	1.3	8.9	2.8	6.1	rain

- 동일한 시각화를 특정 필드 기준으로 **데이터를 쪼개서** 여러 개를 그린다.
 - 인코딩: 인코딩이 같은 시각화가 여러 개가 생긴다.
 - 데이터: 특정 필드로 데이터를 나누기 때문에 각 시각화는 데이터 중 일부를 중복없이 보여준다.



```
alt.Chart(weather).mark_bar().transform_filter(  
    'datum.location == "Seattle"'  
).encode(  
    alt.X('temp_max:Q', bin=True, axis=alt.Axis(title='Temperature (°C)'),  
    alt.Y('count():Q')  
)
```

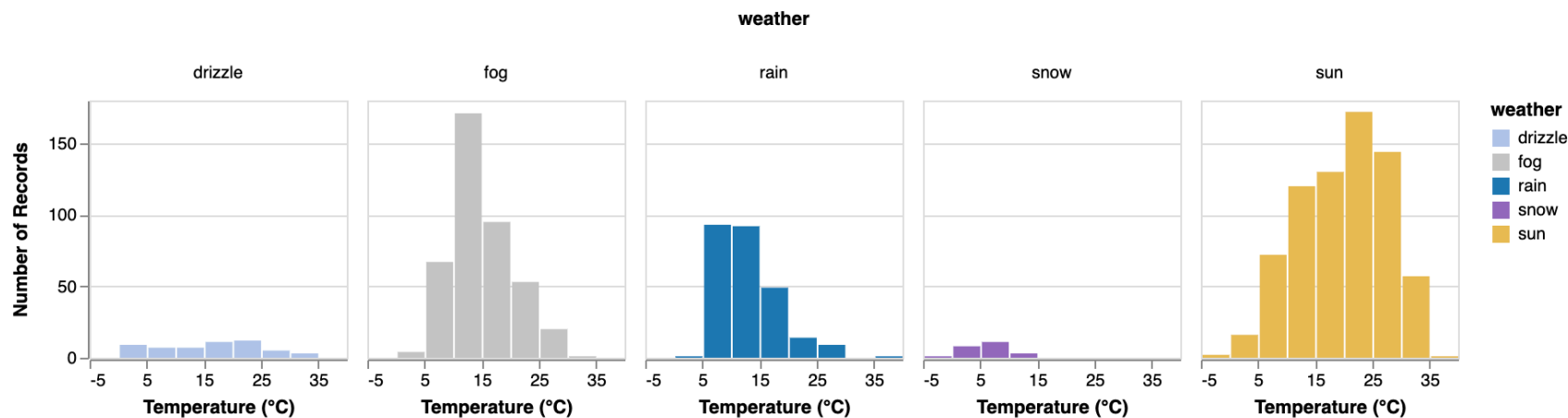


```

colors = alt.Scale(
    domain=['drizzle', 'fog', 'rain', 'snow', 'sun'],
    range=['#aec7e8', '#c7c7c7', '#1f77b4', '#9467bd', '#e7ba52']
)

alt.Chart(weather).mark_bar().encode(
    alt.X('temp_max:Q', bin=True, axis=alt.Axis(title='Temperature (°C)'),),
    alt.Y('count():Q'),
    alt.Color('weather:N', scale=colors)
).properties(
    width=150,
    height=150
).facet(
    column='weather:N'
).transform_filter(
    'datum.location == "Seattle"'
)

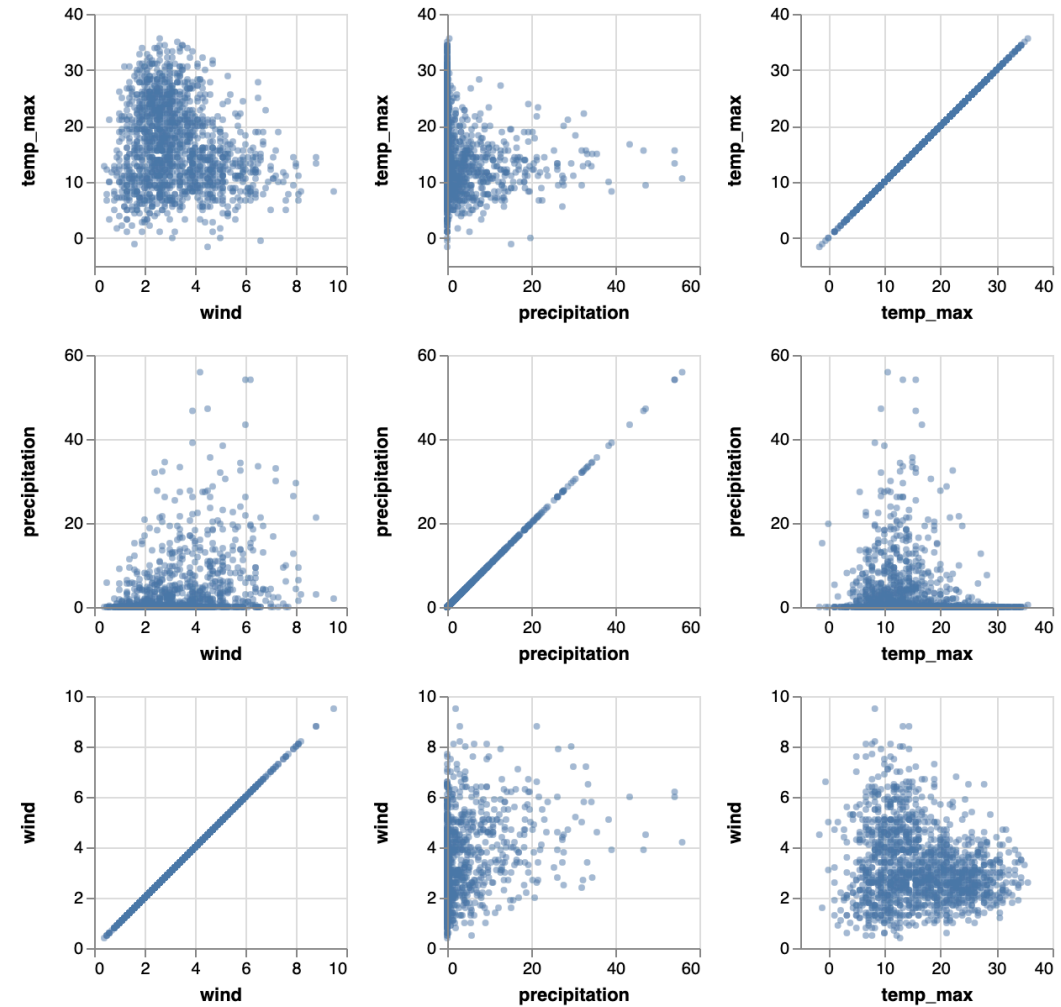
```



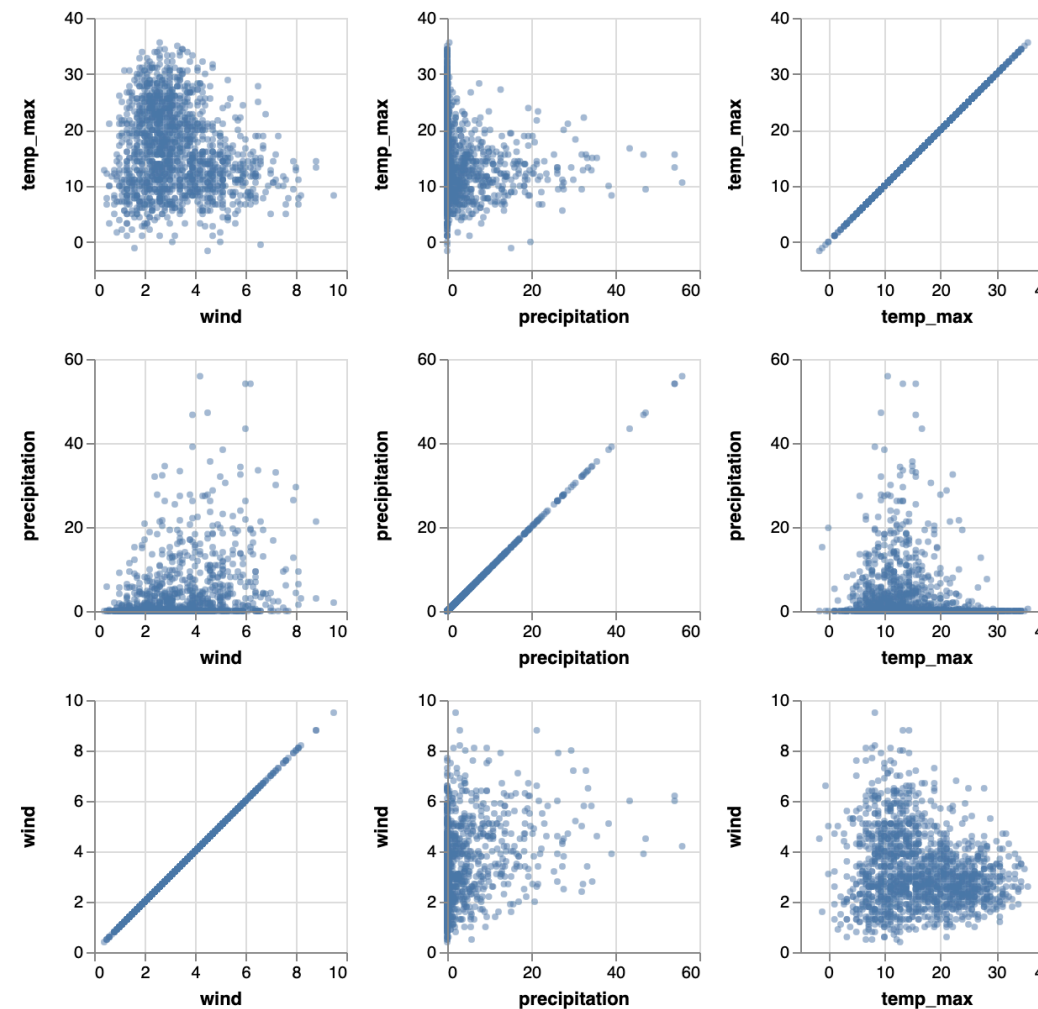
Composition

Repeat

- 인코딩을 바꿔가며 시각화를 여러 개 그린다.
 - **인코딩**이 다른 시각화가 여러 개가 생긴다
(x, y축에 들어가는 변수가 달라짐)
 - 각 시각화는 모든 데이터를 보여준다.



```
alt.Chart(weather).mark_point(
    filled=True, size=15, opacity=0.5).encode(
    alt.X(alt.repeat("column"), type='quantitative'),
    alt.Y(alt.repeat("row"), type='quantitative')
).properties(
    width=150,
    height=150
).repeat(
    row=['temp_max', 'precipitation', 'wind'],
    column=['wind', 'precipitation', 'temp_max']
).transform_filter(
    'datum.location == "Seattle"'
)
```



Repeat vs Facet

- Repeat은 **데이터를 유지**한 채 서로 다른 변수(인코딩)을 바꿉니다
→ 변수간 관계
- Facet은 **인코딩을 유지**한 채 데이터를 쪼개서 여러 개의 시각화를 병치합니다.
→ 라벨(범주)별 차이